

«Компьютерные сети»

(Раздел 2: комментарии к слайдам)

Слайд 3

Введение в Internet

Глобальная сеть Internet реализована на основе стека сетевых протоколов TCP/IP, обеспечивающих передачу данных между разнородными локальными и территориальными сетями (подсетями), а также коммуникационными системами и устройствами.

Краткая история

Появлению сети Internet и стека протоколов TCP/IP предшествовала в середине 1960-х годов разработка под эгидой агентства DARPA (Defence Advanced Research Projects Agency – Управление перспективных исследований Министерства обороны США) сети, получившей название ARPANET (Advanced Research Projects Agency NETwork). Разработка сети была поручена Стэнфордскому исследовательскому институту и трём американским университетам: Калифорнийскому в Лос-Анжелесе и Университетам штата Юта и штата Калифорния в Санта-Барбаре. Экспериментальная сеть из четырёх узлов была запущена в конце 1969 года, а к концу 1972 года в сети насчитывалось более 30 узлов.

В 1974 году были разработаны модели и протоколы TCP/IP для управления обменом данными в интерсетях, а 1 января 1983 года сеть ARPANET полностью перешла на протокол TCP/IP.

В конце 1970-х годов Национальный научный фонд США (National Science Foundation, NSF) начал разработку межуниверситетской сети, получившей название NSFNet, которая имела гораздо бóльшую пропускную способность, чем ARPANET. В середине 1980-х годов произошло объединение сетей NSFNet и ARPANET, за которым закрепилось название INTRNET (Интернет).

В 1984 году была разработана система доменных имён (Domain Name System, DNS), а в 1989 году появилась концепция Всемирной паутины (World Wide Web, WWW) и были разработаны протокол передачи гипертекста HTTP (HyperText Transfer Protocol) и язык разметки гипертекста HTML (HyperText Markup Language).

Интернет объединил большинство существующих сетей и к началу 21 века стал популярным средством для обмена данными.

В настоящее время подключиться к Интернету можно через спутники связи, радиоканалы, кабельное телевидение, телефон, сотовую связь, специальные оптоволоконные линии или электропровода.

Специфические особенности глобальной сети Internet

1. Неограниченный территориальный охват.

2. Сеть объединяет подсети (в том числе локальные сети) разных технологий и компьютеры разных классов (от персональных до суперкомпьютеров).

3. Для передачи данных на большие расстояния используется аппаратура передачи данных (модемы, модемные терминалы) и активное сетевое оборудование (маршрутизаторы, коммутаторы).

4. Топология глобальных сетей, в общем случае, произвольная многосвязная.

5. Одна из важнейших задач – организация эффективной маршрутизации передаваемых данных.

6. Глобальная сеть содержит каналы связи разных типов с пропускными способностями до сотен Гбит/с: кабельные оптические и электрические, в том числе телефонные, беспроводные наземные и спутниковые каналы.

Достоинства сети Internet

1. Неограниченный доступ к любым вычислительным и информационным ресурсам, а также множеству специфических услуг (электронная почта, голосовая связь, конференцсвязь, телевидение по запросу и т.д.).

2. Возможность доступа к ресурсам сети практически из любой точки Земного шара.

3. Возможность передачи любых видов данных, в том числе мультимедийных (аудио, видео).

Слайд 4

Архитектурная концепция

Структура сети Internet может быть представлена как множество компьютеров, называемых **хостами**, подключенных к некоторой единой интерсети, представляющей собой совокупность физических сетей, называемых **подсетями**. В качестве подсетей могут выступать локальные сети, работающие под управлением некоторых аппаратно зависимых протоколов (Ethernet, Token Ring), или коммуникационные системы произвольной физической природы (модемные коммутируемые или выделенные линии, сети X.25, Frame Relay, FDDI, ATM и др.). При этом все функции протокола IP выполняют хосты и маршрутизаторы, называемые узлами сети.

Слайд 5

Стек протоколов TCP/IP

Под **стеком (семейством) протоколов TCP/IP** в широком смысле обычно понимают весь набор реализаций стандартов RFC (Request for Comments).

Основным протоколом стека TCP/IP является протокол IP, который обеспечивает:

- *негарантированную доставку* пакетов, т.к. передаваемые по сети пакеты могут быть утеряны, дублированы, задержаны, доставлены с нарушением порядка;

- *дейтаграммную доставку без установления соединения*, то есть каждый пакет представляет собой обрабатываемый независимо от других блок данных, причем последовательно исходящие от отправителя пакеты могут распространяться по различным путям в сети, менять порядок и даже теряться;

- *максимально возможную доставку* пакетов в том смысле, что потеря пакета происходит лишь в той ситуации, когда протокол не находит никаких физических средств для его доставки.

Соответствие уровней TCP/IP уровням OSI-модели и используемые на каждом уровне основные протоколы стека TCP/IP представлены на слайде.

Модель стека протоколов TCP/IP содержит 4 уровня.

На первом уровне (**Network interface – сетевой интерфейс**) находится аппаратно зависимое программное обеспечение, реализующее передачу данных в той или иной среде. Среда передачи данных может быть реализована различными способами: от простого двухточечного звена до сложной многоузловой коммуникационной структуры сети X.25 или Frame Relay. Стек протоколов TCP/IP поддерживает все стандартные протоколы физического и канального уровней различных сетевых технологий: Ethernet, Token Ring, FDDI, PPP и другие.

На втором уровне (**Internet – межсетевой**) реализуется задача маршрутизации с использованием протокола IP. Вторая важная задача протокола IP – сокрытие аппаратно-программных особенностей среды передачи данных и предоставление вышележащим уровням единого унифицированного и аппаратно независимого интерфейса для доставки данных, что обеспечивает многоплатформенное применение приложений, работающих под TCP/IP.

На третьем уровне (**Transport – транспортный**) решаются задачи надежной доставки пакетов и сохранение их порядка и целостности.

На четвертом уровне (**Application – прикладной**) находятся прикладные задачи, запрашивающие сервис у транспортного уровня.

Основными особенностями стека протоколов TCP/IP являются:

- *независимость от среды передачи* данных;
- негарантированная доставка пакетов.

Информационные объекты (данные), передаваемые на разных уровнях в сети Интернет, получили следующие наименования:

- *сообщение (message)* – блок данных, которым оперирует прикладной уровень, передаваемый от приложения к транспортному уровню с соответствующими этому приложению размером и семантикой;

- *сегмент (segment)* – блок данных, которым оперирует протокол TCP на транспортном уровне;

- *дейтаграмма (datagram)* – блок данных, которым оперирует протокол UDP на транспортном уровне;

- **пакет (packet)** – блок данных, называемый также IP-дейтаграммой, которым оперирует протокол IP на межсетевом уровне;
- **кадр (frame)** – аппаратно зависимые блоки данных, полученные в результате упаковки IP-дейтаграмм в формат, приемлемый для данной физической среды передачи данных и передаваемый на нижнем уровне TCP/IP-модели, называемом «сетевым интерфейсом».

Рассмотрим кратко основные протоколы стека TCP/IP, некоторые из которых (UDP, TCP, IP, ICMP, DHCP, ARP, RIP, OSPF, SLIP, HDLC, PPP) более подробно рассматриваются ниже.

Слайд 6

Протоколы стека TCP/IP прикладного уровня 4

На прикладном уровне реализуются процессы, связанные с выполнением **прикладных программ**, которые можно разбить на 2 группы: **пользовательские**, обеспечивающие решение конкретных пользовательских задач, и **системные**, предоставляющие пользователям множество стандартных услуг, например: обмен файлами, организация электронной почты, контроль и управление сетевыми устройствами и приложениями в сети и т.п. Системные приложения называются **протоколами прикладного уровня** и имеют конкретные названия и соответствующую аббревиатуру.

В зависимости от вида предоставляемых услуг и требований к их реализации прикладные программы в качестве транспорта используют протоколы транспортного уровня: TCP или UDP.

Множество протоколов прикладного уровня можно разбить на классы в зависимости от предоставляемых услуг на протоколы:

- передачи файлов (FTP, TFTP, HTTP);
- почтовые (SMTP, POP3, IMAP);
- контроля и управления сетью (SNMP, DHCP, SIP, DNS);
- прочие (TELNET, BGP, RTP, ...).

Протоколы передачи файлов

FTP (File Transfer Protocol) – протокол передачи файлов, предназначенный для передачи файлов в сети и доступа к удалённым хостам, реализует следующие функции:

- подключение к серверам FTP;
- просмотр содержимого каталогов;
- загрузка файлов с сервера или на сервер.

FTP функционирует поверх транспортного протокола TCP и использует порт 20/TCP для передачи данных и порт 21/ TCP для передачи команд. В протоколе FTP предусмотрены возможности аутентификации и передачи файла с прерванного места, если передача файла была прервана по какой-то причине.

TFTP (Trivial File Transfer Protocol – простой протокол передачи файлов) предназначен главным образом для первоначальной загрузки бездисковых

рабочих станций. TFTP использует транспортный протокол UDP и порт 69/UDP. В отличие от FTP, протокол TFTP не содержит возможностей аутентификации, хотя возможна фильтрация по IP-адресу.

HTTP (HyperText Transfer Protocol – протокол передачи гипертекста) предназначен для передачи данных (изначально – в виде гипертекстовых документов) на основе клиент-серверной технологии. HTTP в настоящее время используется во Всемирной паутине для получения информации с веб-сайтов.

Почтовые протоколы

SMTP (Simple Mail Transfer Protocol) – простой протокол передачи почты, предназначенный для передачи электронной почты в сетях TCP/IP.

POP3 (Post Office Protocol Version 3) – протокол почтового отделения, версия 3, обычно используемый почтовым клиентом в паре с протоколом SMTP для получения сообщений электронной почты с сервера. Протокол POP3 использует транспортный протокол TCP и порт 110/TCP. Альтернативным протоколом для сбора сообщений с почтового сервера является протокол IMAP.

IMAP (Internet Message Access Protocol) – протокол доступа к электронной почте Интернета, как и POP3, служит для работы со входящими письмами, однако обеспечивает ряд дополнительных функций, предоставляя пользователю доступ к хранилищу электронных писем на сервере так, как будто эти письма находятся на его компьютере. POP3 использует транспортный протокол TCP и порт 143/TCP. Для отправки писем используется протокол SMTP.

Протоколы контроля и управления сетью

SNMP (Simple Network Management Protocol – протокол простого управления сетями) предназначен для управления и контроля за сетевыми устройствами и приложениями в сети передачи данных путём обмена управляющей информацией. Протокол SNMP встроен во все сетевые ОС и использует транспортный протокол UDP и порты 161/UDP и 162/UDP.

DHCP (Dynamic Host Configuration Protocol – протокол динамической конфигурации узла) предназначен для автоматического распределения между компьютерами IP-адресов и конфигурационных параметров, необходимых для работы в сети TCP/IP. Протокол реализуется в так называемом DHCP-сервере по клиент-серверной технологии путём выдачи IP-адреса и конфигурационных параметров в ответ на поступивший запрос от компьютера. Протокол DHCP использует транспортный протокол UDP и порты 67/UDP и 68/UDP.

DNS (Domain Name System – система доменных имён) представляет собой компьютерную распределённую иерархическую систему для получения информации о доменах, чаще всего для получения IP-адреса по символьному имени хоста (компьютера или устройства). Распределённая база данных DNS поддерживается с помощью иерархии DNS-серверов, взаимодействующих по одноимённому протоколу. Протокол DNS встроен во все сетевые ОС и использует транспортные протоколы TCP и UDP и, соответственно, порты 53/TCP и 53/UDP.

SIP (Session Initiation Protocol) – протокол установления сеанса, предназначенный для установления и завершения пользовательского интернет-сеанса, включающего обмен мультимедийным содержимым (видео- и аудиоконференции, онлайн-игры).

Прочие протоколы

TELNET (TELEtype NETwork) – виртуальный текстовый терминал, предназначенный для реализации текстового интерфейса в сети с использованием транспортного протокола TCP (стандартный порт 23/TCP).

BGP (Border Gateway Protocol – протокол граничного шлюза) – основной протокол динамической маршрутизации в Интернете, предназначенный для обмена информацией о маршрутах между автономными системами. Функционирует поверх протокола транспортного уровня TCP и использует порт 179/TCP.

RTP (Real-time Transport Protocol) – протокол реального времени, предназначенный для передачи трафика реального времени. Заголовок RTP-пакета содержит данные, необходимые для восстановления голоса или видеоизображения в приёмном узле, о типе кодирования информации (JPEG, MPEG и т. п.) а также временную метку и номер пакета. Эти параметры позволяют при минимальных задержках определить порядок и момент декодирования каждого пакета, а также интерполировать потерянные пакеты. В качестве нижележащего протокола транспортного уровня, *как правило, используется протокол UDP.*

Слайд 7

Протоколы стека TCP/IP уровней 1-3

К протоколам стека TCP/IP уровней 1-3 относятся:

- транспортные протоколы уровня 3 (TCP, UDP и др.);
- протоколы межсетевого уровня 2 (IP, ISMP, IGMP, RIP, OSPF, ARP, RARP и др.);
- протоколы сетевого доступа уровня 1 (SLIP, HDLC, PPP).

Протоколы транспортного уровня

TCP (Transmission Control Protocol) – протокол управления передачей данных с установлением соединения, реализующий обмен данными между двумя узлами на основе некоторого соглашения об управлении потоком данных.

UDP (User Datagram Protocol) – дейтаграммный протокол передачи данных в виде независимых единиц – дейтаграмм (datagram).

Протоколы межсетевого уровня

IP (Internet Protocol) - основной протокол стека TCP/IP, реализующий передачу пакетов по IP-сети от узла к узлу без гарантии доставки пакетов, сохранения целостности пакетов и порядка потока пакетов.

ICMP (Internet Control Message Protocol) – межсетевой протокол управляющих сообщений, используемый в основном для передачи сообщений об ошибках и исключительных ситуациях, возникших при передаче данных, а также выполняющий некоторые сервисные функции.

ICMP-сообщения генерируются при нахождении ошибок в заголовке IP пакета, при отсутствии маршрута к адресату, а также используются маршрутизаторами для обновления записей в таблице маршрутизации отправителя и для управления скоростью отправки сообщений отправителем. ICMP-сообщения инкапсулируются в IP пакеты.

RIP (Routing Information Protocol) – протокол маршрутизации типа DVA, реализующий алгоритм обмена информацией о доступных сетях и расстояниях до них путём *периодической* рассылки широковещательных пакетов.

OSPF (Open Shortest Path First) – протокол маршрутизации типа LSA, реализующий алгоритм обмена информацией о состоянии каналов, путём периодического тестирования состояния каналов с соседними маршрутизаторами. Протокол OSPF разработанный для применения в сети Интернет и используется в других больших сетях (NetWare, SNA, XNS, DECNet).

ARP (Address Resolution Protocol – Протокол разрешения адресов) предназначен для определения физического адреса устройства (MAC-адреса) по его IP-адресу.

RARP (Reverse Address Resolution Protocol – Протокол обратного определения адреса) предназначен для определения IP-адреса устройства по его физическому адресу (MAC-адресу).

Протоколы **ARP** и **RARP** часто рассматривают как протоколы промежуточного уровня 2,5 *OSI-модели*.

Протоколы канального уровня («сетевой интерфейс»)

SLIP (Serial Line IP) – первый стандарт канального уровня для выделенных линий, разработанный специально для стека протоколов TCP/IP, который благодаря простоте может использоваться как для коммутируемых, так и для выделенных каналов. SLIP поддерживается только протоколом сетевого уровня IP.

HDLC (High-level Data Link Control Procedure) – высокоуровневый протокол управления каналом – стандарт ISO для выделенных линий, представляющий собой семейство протоколов LAP (Link Access Protocol). HDLC относится к бит-ориентированным протоколам.

PPP (Point-to-Point Protocol) – протокол двухточечного соединения, пришедший на смену протоколу SLIP и построенный на основе формата кадров протоколов семейства HDLC с дополнением собственных полей. PPP является стандартным протоколом Интернета и так же, как протокол HDLC, представляет собой семейство протоколов.

Слайд 8

Концептуальная модель передачи данных в сетях TCP/IP

На слайде иллюстрируется концептуальная модель передачи данных в сетях TCP/IP.

В хостах реализуются протоколы всех четырех уровней модели стека TCP/IP, а в маршрутизаторах – второго и третьего уровней.

Слайд 9

Способы передачи данных

Поток передаваемых данных (трафик) в компьютерных сетях делится на четыре вида:

- **unicast** (индивидуальный) – поток данных, предназначенных только одному адресату;
- **multicast** (групповой) – поток данных, предназначенных ограниченной группе узлов (адресатов);
- **broadcast** (широковещательный) – поток данных, предназначенных всем узлам (адресатам) в пределах одной локальной сети (ограниченный широковещательный поток); при необходимости возможен вариант передачи широковещательного трафика в соседнюю локальную сеть (направленный широковещательный поток) за счет специального конфигурирования маршрутизатора, объединяющего эти сети, позволяющего пропускать широковещательные пакеты;
- **anycast** (любой) – поток данных, предназначенных нескольким идентичным узлам, например DNS-серверам, при этом пакет будет доставлен ближайшему узлу (DNS-серверу).

Слайд 10

Типы адресов

В стеке протоколов TCP/IP используются три типа адресов:

- **физические (локальные) адреса**, используемые для адресации узлов (компьютеров, маршрутизаторов, коммутаторов, ...), примерами которых могут служить: MAC-адреса, если подсеть использует технологии Ethernet, Token Ring, FDDI, или IPX-адреса, если подсеть на основе технологии IPX/SPX;
- **сетевые (IP-адреса)**, используемые для идентификации узлов в пределах всей составной сети (подсети);
- **доменные имена** – символьные идентификаторы узлов, которыми оперируют пользователи.

Длина локального MAC-адреса, используемого наиболее широко из-за повсеместного распространения локальных сетей Ethernet, равна 6-и байтам.

При разработке стандарта локальной сети Ethernet было принято, что каждая сетевая карта и каждый встроенный сетевой интерфейс должны иметь уникальный номер – MAC-адрес, присвоенный при изготовлении.

Уникальность MAC-адреса достигается тем, что адреса назначаются *централизованно* комитетом IEEE, который распределяет между производителями интерфейсных карт (сетевых адаптеров) так называемые организационно уникальные идентификаторы (Organizationally Unique Identifier, OUI), отображаемые в первых трех байтах адреса, а в следующих трех байтах помещается порядковый номер сетевого адаптера, присваиваемый производителем.

При этом в первом байте адреса имеются два бита, определяющие тип и способ назначения адреса:

- **нулевой бит** (бит 0) указывает *тип адреса* (U/M – Unicast/Multicast): **0** – адрес назначения является **уникальным**, **1** – адрес назначения является **групповым** (тогда в последующих разрядах указывается адрес конкретной группы), или **широковещательным**, если все остальные разряды равны 1, то есть кадр адресован всем хостам в локальной сети;

- **первый бит** (бит 1) указывает *способ назначения адреса* (G/L – Global/Local): **0** – адрес является **глобальным** физическим адресом; **1** – адрес **локальный**, т.е. назначен администратором ЛВС и используется только в пределах этой сети.

Существует две формы записи MAC-адреса (форматы): **каноническая** и **бит-реверсная**. Последняя отражает последовательность передачи *битов* каждого байта по каналу связи.

Слайд 11

Классовая адресация

IP-адрес – идентификатор *сетевого соединения (сетевого интерфейса)*. Это означает, что один и тот же компьютер, соединенный с двумя сетями, имеет два IP-адреса: сеть 1 идентифицирует его по адресу IP1, а сеть 2 – по адресу IP2.

IP-адреса представляют собой 32-битовые идентификаторы, ориентированные на решение основной задачи протокола IP – маршрутизации. Для удобства представления IP-адресов используется цифровое их написание в виде 4-х десятичных чисел, разделенных точками, например: **192.171.153.60** .

Первоначально в Интернете была принята так называемая классовая адресация. Все IP-адреса разделены на 5 классов (от А до Е), но практическое применение находят в основном три первых класса: А, В и С. Класс D предназначен для задания группового адреса, а класс Е – не используется (был зарезервирован).

IP-адрес состоит из двух полей: поле «Номер сети», представляющий собой адрес физической сети (подсети), и поле «Номер узла», выделяющий в этой подсети конкретное устройство (хост).

Признаком принадлежности адреса к определённому классу служат первые биты адреса: если первый бит равен 0, то адрес принадлежит классу А, если первый бит равен 1, а второй – 0, то адрес принадлежит классу В и т.д.

Принадлежность адреса к тому или иному классу определяет размер сети:

- класс А соответствует большой сети с максимальным числом узлов ($2^{24} - 2$) = 16 777 214;
- класс В соответствует средней сети с числом узлов до 65534;
- класс С соответствует малой сети с числом узлов до $(2^8 - 2) = 254$.

Отметим, что **максимальное количество узлов** в сети определяется количеством двоичных разрядов n , отводимых под номер узла: $N_{\max} = 2^n - 2$, то есть исключаются два номера:

- **нулевой** (все разряды равны 0); адрес с нулевым значением номера узла означает *адрес сети*;
- **единичный** (все разряды равны 1); адрес с единичными значениями номера узла является *широковещательным* и означает передачу пакета всем узлам сети.

IP-адрес построен таким образом, чтобы поля «Номер сети» и «Номер узла» можно было бы выделить быстро, что особенно сказывается на эффективности маршрутизации (малые временные затраты на выделение адреса «Номер сети»).

Поскольку IP-адрес идентифицирует сетевое соединение, а не узел, то отсюда вытекает принципиальное ограничение: *если компьютер переносится из одной подсети в другую, он должен обязательно изменить IP-адрес.*

Слайд 12

Специальные адреса и маскирование

IP-адресация поддерживает **специальные адреса**, обращенные к множеству узлов и/или сетей и делящиеся на два класса:

- широковещательные (broadcast), обращенные ко всем;
- групповые (multicast), обращенные к заданному множеству объектов.

Адресная нотация при этом основывается на заполнении адресных полей нулями (обращение к данному объекту, this) или единицами (обращение ко всем объектам, all).

К специальным адресам относятся следующие.

Адрес 1 – "пустышка" или **неопределённый адрес**, используемый в инициализационной процедуре, когда рабочая станция не знает своего IP-адреса или хочет его согласовать; используется только как *адрес отправителя*, но никогда как адрес получателя.

Адрес 2 – адрес конкретного узла (XXX...X) в той же сети, что и узел-отправитель; применяется в случае, когда узел-отправитель не знает идентификатора сети, в которой работает, например при инициализации бездисковой рабочей станции, которая при включении вообще ничего не знает

ни о сети, ни о себе; используется только как *адрес получателя* и никогда как адрес отправителя.

Адрес 3 – адрес сети (но не узла).

Адрес 4 – **локальный** или **ограниченный широковещательный** адрес (limited или local broadcast address); используется, когда идентификатор сети по каким-либо причинам неизвестен; для использования не рекомендуется.

Адрес 5 – **прямой широковещательный** адрес (direct broadcast address), обращенный ко всем узлам данной сети.

Адрес 6 – **тестовый адрес**, в котором первый байт имеет значение 127, а оставшееся поле не специфицировано; используется для задач отладки и тестирования, не является адресом никакой сети, и маршрутизаторы никогда не обрабатывают его; также называется **адресом обратной петли** (loopback address), поскольку пакет с таким адресом, посланный на интерфейс loopback возвращается на тот же интерфейс, не выходя за пределы подсети.

Интерфейс loopback имеет несколько применений. Он может быть использован сетевым клиентским программным обеспечением, чтобы общаться с серверным приложением, расположенном на том же компьютере. Этот механизм полезен для тестирования служб, не подвергая их безопасности риску, как при удаленном сетевом доступе.

В стандартах Интернета определено несколько так называемых **автономных адресов**, рекомендуемых для автономного использования в пределах одной подсети и необрабатываемых маршрутизаторами:

- в классе А: 10.0.0.0 (1 сеть);
- в классе В: 172.16.0.0 – 172.31.0.0 (16 сетей);
- в классе С: 192.168.0.0 – 192.168.255.0 (256 сетей).

В качестве **группового адреса** используются адреса класса D. Групповая адресация в TCP/IP регламентируется входящим составной частью в IP протоколом *IGMP (Internet Group Management Protocol)*. Групповой адрес может объединять узлы из разных физических сетей путем использования в маршрутизаторах специальных протоколов групповой маршрутизации. Каждый узел может в любой момент подключиться к определенной адресной группе или выйти из нее.

Групповые адреса назначаются NIC и разделяются на два класса:

- **постоянные** – для непрерывно существующих групп (так называемые «всем известные адреса» – well-known addresses);
- **временные** – для организуемых на некоторый срок групп, которые существуют до тех пор, пока в группе сохраняется хотя бы один член (так называемые «временные адресные группы» – transient multicast groups).

Распространение групповых сообщений по сети ограничивается временем жизни TTL (time-to-live) IP-пакета.

Использование масок для IP-адресов

Маска представляет собой 32-разрядный двоичный код, содержащий в *нескольких первых (старших) разрядах «единицы»*, а в остальных – «нули». Количество единиц в маске определяет границу номера (идентификатора) сети. Другими словами, **единичные значения маски позволяют выделить из IP-адреса номер сети**, а оставшиеся младшие разряды **IP-адреса определяют номер узла в этой сети**.

Использование масок для IP-адресов позволяет *расширить адресное пространство* и сделать систему адресации более гибкой, не привязанной к классам **IP-адресов (А, В или С)**.

Пример использования маски показан на слайде.

Часто использование маски указывается в виде: 116.165.42.35/**12**, где число **12** определяет количество единичных разрядов в маске для IP-адреса 116.165.42.35.

Слайд 13

Технология бесклассовой междоменной маршрутизации CIDR (Classless Inter-Domain Routing)

Использование масок переменной длины для IP-адресов позволяет не только расширить адресное пространство за счет увеличения количества номеров сетей, но и экономно выделять IP-адреса.

Например, если в какой-то небольшой сети находится десяток узлов, то очевидно, что неразумно выделять ей номер сети даже класса С, обеспечивающей нумерацию 254-х узлов. Гораздо более эффективным будет выделение для этой сети небольшого количества IP-адресов.

Для выделения ограниченного количества IP-адресов разработана технология **бесклассовой междоменной маршрутизации (CIDR – Classless Inter-Domain Routing)**, использующая *бесклассовую адресацию* и позволяющая гибко распределять IP-адреса.

Для реализации технологии CIDR необходимо, чтобы организация, распределяющая IP-адреса, имела в наличии непрерывный диапазон адресов. Это предоставляет возможность выделять сетям некоторое количество IP-адресов, имеющих *одинаковый префикс*, то есть одинаковые значения в нескольких старших разрядах.

На слайде показан пример, иллюстрирующий принцип выделения адресов из общего пула адресов для сетей разных размеров. Так, например:

- пул Р1 имеет префикс длиной 28 двоичных разрядов и 4 разряда под нумерацию узлов, что позволяет пронумеровать 16 узлов небольшой сети;
- пул Р2 имеет префикс длиной 26 разрядов и 6 разрядов под нумерацию узлов, что позволяет пронумеровать 64 узла;
- пул Р3 с префиксом длиной 26 разрядов позволяет пронумеровать 1024 узла.

При таком выделении адресов необходимо, чтобы:

- количество выделяемых адресов было кратно степени двойки;
- начальная граница выделяемого пула адресов была кратна требуемому количеству узлов.

Благодаря технологии CIDR имеется возможность нарезать блоки адресов в соответствии с действительными потребностями каждой сети.

Слайд 14

Формирование подсетей в большой сети

Технология CIDR, использующая маскирование сетевых адресов, позволяет большую сеть некоторой организации разбить на подсети, каждая из которых объединяет, например, пользователей одного подразделения (отдела, лаборатории).

Очевидно, что количество подсетей и хостов в каждой подсети с заданной длиной маски зависит от размера исходной большой сети.

В таблице представлены варианты формирования подсетей одинакового размера, число которых зависит от количества хостов в подсети.

Следует отметить, что общее количество хостов в сети уменьшается с увеличением количества подсетей.

Слайд 15

Подсети разных размеров в рамках сети 185.68.1.0/24

Большая сеть может быть разбита на подсети разного размера.

На слайде иллюстрируется один из возможных вариантов разбиения большой сети 185.68.1.0/24 на 6 подсетей, причем все подсети разного размера за счет присвоения им разных масок: от сети со 126 узлами (хостами) с маской /25 до подсети с двумя узлами с маской /30.

Таким образом, в рамках сети 185.68.1.0/24 сформировано 6 подсетей разного размера с разными масками с максимальным суммарным числом узлов, равным 240.

Слайд 16

Технология NAT

NAT (Network Address Translation) – сетевая технология, направленная на решение проблемы ограниченного диапазона IP-адресов в IPv4 и повышение безопасности сети за счет:

- ограничения доступа извне к ресурсам внутренней сети при сохранении возможности выхода в публичную (внешнюю) сеть;
- сокрытия внутренних сервисов, хостов, серверов.

Изменение адреса при передаче пакета из внутренней сети во внешнюю и обратно осуществляется пограничным маршрутизатором, в котором реализована технология NAT. Это изменение выполняется с использованием NAT-таблицы, устанавливающей соответствие между глобальными IP-адресами внешней сети и локальными IP-адресами внутренней сети. В некоторых реализациях NAT-таблица дополнительно может содержать номера портов, привязанных к IP-адресам.

Существует несколько типов технологии NAT, основными из которых являются:

- статический (Static) NAT, когда один локальный адрес соответствует одному глобальному адресу;
- динамический (Dynamic) NAT, когда локальному адресу соответствуют разные глобальные адреса;
- Port Address Translation (PAT) или NAT Overload (перегруженный NAT), когда сопоставление локальных и глобальных адресов осуществляется с использованием портов.

Статический NAT обычно применяется для отображения незарегистрированного IP-адреса на зарегистрированный IP-адрес и используется в сети, когда необходимо, чтобы IP-адрес всегда был доступен из глобальной сети.

Динамический NAT отображает незарегистрированный IP-адрес внутренней сети на зарегистрированный адрес из пула зарегистрированных IP-адресов внешней сети, причем зарегистрированный адрес может меняться в зависимости от доступного в данный момент IP-адреса в пуле адресов.

Перегруженный NAT (NAT Overload) или PAT динамически отображает несколько незарегистрированных IP-адресов в один зарегистрированный IP-адрес, используя при этом различные порты.

Слайд 17

Распределение IP-адресов

Способы распределения IP-адресов:

- **централизованное распределение**, реализуемое в соответствии с иерархией, показанной на слайде;
- **автоматизированное распределение** (назначение компьютерам) IP-адресов внутри подсети на основе протокола DHCP.

DHCP (Dynamic Host Configuration Protocol) – протокол динамической конфигурации хостов, предназначенный для автоматического получения сетевыми устройствами (компьютерами) IP-адреса и некоторых дополнительных параметров, необходимых для работы в сети TCP/IP.

Протокол DHCP предоставляет три способа распределения IP-адресов.

1. **Ручное статическое распределение**, при котором сетевой администратор назначает физическому адресу (MAC-адресу в сетях Ethernet) клиентского компьютера определённый IP-адрес. При этом роль DHCP-сервера сводится к централизованному хранению сведений об адресах.

2. **Автоматическое статическое распределение**, при котором компьютеру предоставляется на постоянное использование свободный IP-адрес из имеющегося диапазона адресов.

3. **Автоматическое динамическое распределение**, которое отличается от предыдущего способа лишь тем, что IP-адрес выдаётся компьютеру на определённый срок – *срок аренды адреса*, по истечении которого адрес считается свободным, а клиент при необходимости должен запросить новый адрес. Таким образом, один и тот же IP-адрес в разные интервалы времени может быть предоставлен разным компьютерам.

Для корректной работы в сети компьютеру, кроме IP-адреса, требуются другие конфигурационные параметры, такие как:

- маска подсети;
- IP-адрес маршрутизатора по умолчанию;
- IP-адреса DNS-серверов;
- доменное имя компьютера и т.д.

Эти параметры включаются в DHCP-сообщение и передаются вместе с IP-адресом.

Слайд 18

Протокол DHCP

Протокол DHCP был принят в 1993 году и в настоящее время используется в большинстве сетей TCP/IP.

Протокол DHCP работает на *прикладном уровне* OSI-модели по схеме «клиент-сервер», используя порт 67/UDP или 68/UDP. Для автоматической конфигурации компьютер-клиент при включении обращается к серверу DHCP и получает от него IP-адрес и ряд конфигурационных параметров.

DHCP-сервер – это компьютер, который распределяет IP-адреса и следит за тем, чтобы один и тот же IP-адрес не был выдан двум клиентам. Для получения IP-адреса клиент и сервер обмениваются между собой DHCP-сообщениями в режиме запрос-ответ. Назначение DHCP-сообщений представлено в таблице на слайде.

Получение IP-адреса выполняется за 4 шага.

1. Компьютер клиента для нахождения DHCP-сервера формирует сообщение DISCOVER с широковещательным MAC-адресом FF:FF:FF:FF:FF:FF, которое принимают все компьютеры в сети. Использование широковещательного адреса обусловлено тем, что клиент при включении не знает, где находится DHCP-сервер.

2. В ответ DHCP-сервер посылает клиенту сообщение OFFER с IP-адресом.

3. Клиент подтверждает получение IP-адреса, посылая сообщение REQUEST с тем же IP-адресом.

4. DHCP-сервер в ответ посылает подтверждение ACK о возможности использования клиентом данного IP-адреса.

Такую процедуру получения IP-адреса по протоколу DHCP обозначают аббревиатурой **DORA** – по первым буквам передаваемых DHCP сообщений: **DISCOVER, OFFER, REQUEST, ACK**.

Кроме перечисленных сообщений могут использоваться и другие сообщения:

- NACK – на запрос клиентом IP-адреса в сообщении REQUEST запрещает его использование;
- RELEASE – клиент освобождает IP-адрес;
- DECLINE – клиент отказывается от предлагаемого IP-адреса;
- INFORM - используется клиентом, имеющим IP-адрес, для получения дополнительных конфигурационных параметров;
- FORCERENEW – требование освободить IP-адрес (практически не используется).

Может возникнуть вопрос: а зачем нужен 4-й шаг? Разве трёх шагов недостаточно?

Такая необходимость в 4-м шаге может возникнуть в следующих ситуациях:

- 1) возникла **ошибка** при передаче IP-адреса от DHCP-сервера клиенту, которая в случае 3-х шагов не будет обнаружена и клиент будет использовать этот ошибочный IP-адрес; наличие 4-го шага позволяет DHCP-серверу выслать запрет на использование IP-адреса в виде сообщения NACK
- 2) в сети имеется **два и более DHCP-сервера**.

Компьютеру IP-адрес выделяется DHCP-сервером на ограниченное время – **время аренды (lease time)**, которое может составлять от нескольких минут до нескольких дней. По истечении времени аренды IP-адрес освобождается и может передан другому компьютеру.

При необходимости аренда IP-адреса может быть продлена клиентом. Для этого клиент по истечении половины времени аренды пересылает серверу сообщение REQUEST, в котором указывает этот IP-адрес. В ответ сервер высылает сообщение ACK, подтверждающее возможность дальнейшего использования этого IP-адреса.

При выключении компьютера операционная система автоматически отправляет DHCP-серверу сообщение RELEASE, извещающее об освобождении IP-адреса, который может быть передан другому компьютеру. При некорректном

выключении компьютера и при сбоях в операционной системе IP-адрес сохраняется за компьютером до тех пор, пока не закончится время аренды.

DHCP-сервер и клиенты, запрашивающие IP-адрес, обычно располагаются в одной и той же подсети. Если же DHCP-сервер находится в другой подсети, отделенной от клиента маршрутизатором, то DHCP-сообщение DISCOVER может не дойти до сервера, поскольку *маршрутизатор не пропускает широковещательный трафик*. Для решения этой проблемы может использоваться специальная конфигурация маршрутизатора – **DHCP-Relay**, которая позволяет пропускать (транслировать) широковещательный трафик, но не весь, а только DHCP-сообщений. В этом случае широковещательное DHCP-сообщение DISCOVER будет доставлено от клиента к DHCP-серверу, который находится в другой подсети, а DHCP-сервер вышлет IP-адрес клиенту.

Слайд 19

Протоколы ARP и RARP (InARP)

Определение физического адреса устройства (MAC-адреса) по его IP-адресу и наоборот, IP-адреса по MAC-адресу, решают три протокола:

- **ARP** (*Address Resolution Protocol*) – протокол разрешения адресов, предназначенный для определения MAC-адреса по IP-адресу;
- **RARP** (*Reverse Address Resolution Protocol*) – протокол обратного определения адреса;
- **InARP** (*Inverse Address Resolution Protocol*) – инверсный протокол для получения сетевых адресов других рабочих станций по адресам канального уровня.

Протокол ARP в режиме запрос-ответ позволяет автоматически определить MAC-адрес компьютера по его IP-адресу. Для этого протокол ARP поддерживает в каждом узле (сетевом адаптере или порту маршрутизатора) **ARP-таблицу**, содержащую:

- IP-адрес;
- MAC-адрес;
- тип записи (динамический, статический).

Статические записи создаются вручную и существуют, пока соответствующий узел (компьютер или маршрутизатор) не будет выключен.

Динамические записи создаются протоколом ARP как по собственным ARP-запросам, так и путем извлечения из широковещательных запросов IP- и MAC-адресов отправителя. Динамическая запись имеет «срок жизни», после которого она удаляется. Срок жизни записи устанавливается администратором сети или принимается по умолчанию и может составлять от десятков секунд до нескольких часов.

По ARP-таблице узел может определить физический адрес (MAC-адрес) узла назначения, находящегося в этой же сети, по известному IP-адресу и указать его в заголовке кадра канального уровня.

Если в ARP-таблице отсутствует запись для некоторого IP-адреса, то узел формирует *широковещательное* сообщение – **ARP-запрос** (ARP-request), в котором запрашивает физический адрес узла назначения с заданным IP-адресом. Все узлы сети принимают этот запрос, однако лишь один узел, IP-адрес которого совпадает с указанным в ARP-запросе, отвечает на него, высылая **ARP-ответ** (ARP-reply), формат которого такой же, как у ARP-запроса, со своим MAC-адресом *непосредственно* узлу, приславшему ARP-запрос. *Отправитель извлекает MAC-адрес и вставляет его в поле «адрес назначения» формируемого для передачи по локальной сети кадра.* Полученный отправителем ARP-запроса MAC-адрес кэшируется в ARP-таблице.

Формат ARP-запроса (ответа), представленный на слайде, содержит 9 полей.

Протокол ARP в модели OSI располагается *между канальным и сетевым уровнем* (уровень 2,5). Пакеты ARP вкладываются непосредственно в **кадры локальной сети** и не передаются через маршрутизаторы, поскольку содержат широковещательный адрес. Следовательно, с помощью протокола ARP можно узнать только MAC-адреса компьютеров, которые находятся в одной подсети, а именно в локальной сети.

Отправитель также включает в ARP запрос свой IP-адрес и свой MAC-адрес, которые могут быть занесены всеми компьютерами сети в ARP-таблицы.

Каждый компьютер локальной сети может формировать специальные ARP-запросы. Примером такого запроса может служить так называемый «**добровольный**» ARP-запрос (*Gratuitous ARP*), с помощью которого можно:

- *оповестить* все компьютеры сети об изменении или появлении у данного компьютера нового IP-адреса;
- *проверить* возможность использования данного IP-адреса другим компьютером: если на такой ARP-запрос пришел ответ, то это значит, что в сети есть компьютер с таким IP-адресом.

Таким образом, ARP-протокол разрешения адресов:

- позволяет по IP-адресу определить MAC-адрес компьютера в сети;
- работает в режиме запрос-ответ;
- запрос отправляется на широковещательный адрес и его получают все компьютеры в сети, а отвечает только тот компьютер, который узнал свой IP-адрес и в ответ он вкладывает искомый MAC-адрес.

Наиболее типичная ситуация, при которой возникает необходимость использования протокола ARP, следующая. В сети, объединяющей несколько локальных сетей (подсетей) с помощью маршрутизаторов, продвижение пакетов от узла, находящегося в одной подсети, к узлу, находящемуся в другой подсети, осуществляется на основе старшей части IP-адреса, то есть на основе номера

сети. После того, как пакет поступит в *конечный маршрутизатор*, к которому подсоединена локальная сеть, в которой находится получатель кадра, необходимо этот пакет упаковать в кадр, а в качестве канального адреса узла назначения указать его MAC-адрес. Маршрутизатор просматривает свою ARP-таблицу и, если не находит соответствующего IP-адреса, формирует широковещательный ARP-запрос, посылает его в локальную сеть и ожидает ARP-ответа. Если в сети нет компьютера с указанным в ARP-запросе IP-адресом, то ARP-ответа не будет, и протокол IP уничтожит все пакеты, направляемые по этому адресу.

Посмотреть свою ARP таблицу на компьютере можно с помощью команды `arp -a`.

RARP (*Reverse Address Resolution Protocol*) — *обратный протокол преобразования адресов*, предназначенный для определения IP-адреса по физическому адресу.

Протокол используется в случае, если узел – бездисковая рабочая станция, у которой только что включили питание и она не только ничего не знает о себе и окружающих, но и не может произвести дистанционную загрузку операционной системы, которая хранится на сетевом диске. В этом случае формируется групповой RARP-запрос со своим физическим адресом (при этом узел может даже не знать адреса сервера). Сервер принимает запрос и просматривает свои таблицы, либо перенаправляет его. В сети находится по меньшей мере один обслуживающий такие запросы сервер (RARP-сервер), который распознает запрос от рабочей станции, выбирает из некоторого списка свободный IP-адрес и шлет этому узлу сообщение с необходимой информацией:

- динамически выделенный узлу IP-адрес;
- свой физический адрес;
- IP-адрес и т.д.

После обнаружения найденный адрес отсылается на запросивший его узел. Другие станции также могут «слышать» этот диалог и локально сохранить эту информацию в своих ARP-таблицах.

Поскольку при таком механизме отказ RARP-сервера очень критичен в том смысле, что без его услуг не заработает целый ряд рабочих станций, то обычно сеть конфигурируется так, чтобы протокол RARP поддерживали несколько серверов в сети.

RARP позволяет разделять IP-адреса между нечасто используемыми хост-узлами. После использования каким-либо узлом IP-адреса он может быть освобождён и выдан другому узлу.

RARP является дополнением к ARP.

InARP (*Inverse Address Resolution Protocol, Inverse ARP*) — протокол, предназначенный для получения сетевых адресов (и *не только IP-адресов*) других рабочих станций по их адресам канального уровня. В основном используется в сетях Frame Relay и ATM.

InARP можно рассматривать как расширение ARP.

Таким образом, RARP, как и InARP, предназначен для определения адреса сетевого уровня по адресу канального уровня. Отличие состоит только в том, что RARP-протокол используется для получения сетевого адреса *самой станции отправителя*, в то время как InARP-протокол запрашивает сетевой адрес по физическому адресу *другой станции*.

Отметим, что ARP запросы и ответы размещаются в поле данных кадра, что позволяет рассматривать протокол ARP как протокол OSI-модели промежуточного уровня 2,5, расположенного между сетевым и канальным уровнями.

Слайд 20

Протокол DNS (Domain Name System)

Доменное имя – символьное имя компьютера.

Совокупность имен, у которых несколько старших составных частей совпадают, образуют **домен (domain) имен**.

В стеке TCP/IP применяется система доменных имен с *иерархической древовидной структурой*, допускающей использование в имени произвольного количества составных частей.

Примерами доменных имён организаций являются:

- com – коммерческие организации;
- edu – образовательные организации;
- gov – правительственные организации;
- org – некоммерческие организации;
- net – организации поддержки сетей.

Соответствие между доменными именами и IP-адресами может устанавливаться как средствами локального узла, так и средствами централизованной службы, реализуемой системой доменных имён.

Система доменных имен DNS – централизованная служба, основанная на распределенной базе отображений «доменное имя – IP-адрес». DNS предназначена для замены цифровых адресов на символьные, более понятные пользователям.

Служба DNS использует в своей работе протокол типа «клиент-сервер», в котором определены такие понятия как **DNS-сервер**, поддерживающий распределенную базу отображений, и **DNS-клиент**, обращающийся к DNS-серверу с запросом. DNS-сервер использует текстовые файлы формата «IP-адрес – доменное имя».

Служба DNS является распределенной. Каждый DNS-сервер хранит имена следующего уровня иерархии и, кроме таблицы отображений имен, содержит ссылки на DNS-серверы своих поддоменов, что упрощает процедуру поиска.

Для ускорения поиска IP-адресов в DNS-серверах применяется кэширование проходящих через них ответов на определенное время – от нескольких часов до нескольких дней.

Иерархическая организация DNS предполагает использование четырех типов DNS-серверов.

1. **Рекурсивный DNS-сервер (Resolver)** – сервер, который формирует запросы к другим DNS-серверам, пока не найдет IP-адрес по запросу DNS-клиента. В качестве ответа возвращает клиенту найденный IP-адрес, либо сообщение об ошибке, если IP-адрес не найден.

Обычно для надежности создаются 2 рекурсивных DNS-сервера: *основной* и *запасной*. Например, основной публичный рекурсивный DNS-сервер от Google имеет IP-адрес 8.8.8.8, а запасной – 8.8.4.4.

2. **Корневой (Root) DNS-сервер** обслуживается *Интернет-корпорацией по управлению доменными именами и IP-адресами* (ICANN - Internet Corporation for Assigned Names and Numbers). В настоящее время в Интернете 13 корневых DNS-серверов (8 в США, 3 в Европе и 2 в Азии), копии которых распределены по всему миру. В России копии есть в Москве, Екатеринбурге и Новосибирске. Основная задача корневых DNS-серверов – поддерживать каталоги для существующих доменных зон. Другими словами, они знают адреса TLD-серверов, отвечающих за конкретные доменные зоны: «.ru», «.com», «.net» и т.д. То есть, если нужно найти IP-адрес домена «itmo.ru», он вернет IP адрес TLD-сервера, отвечающего за зону «.ru».

3. **TLD (Top Level Domain)** – сервер домена верхнего уровня – хранит каталог с адресами *авторитативных* серверов своей зоны и поддерживается *Административным органом по присвоению адресов* – IANA (Internet Assigned Numbers Authority), которая входит в ICANN. TLD-сервер знает на каком авторитативном сервере хранится информация о любом домене из его зоны.

4. **Авторитативный (Authoritative) DNS-сервер** хранит всю информацию о конкретном домене. Это не только IP адрес, но и другие записи разных типов, обозначаемых заглавными буквами:

- А — адрес хоста (IPv4 address);
- AAAA — IPv6 адрес (IPv6 address);
- MX — имена почтовых серверов (Mail eXchange);
- NS — сервер домена (Name Server);
- TXT — текстовые записи (Text), SPF.

Там же хранится информация об организации, которая осуществила регистрацию домена. Эти данные в свободном доступе и их можно получить через онлайн-сервисы.

Авторитативный DNS-сервер:

- быстро реагирует на запросы;
- не отвечает на рекурсивные запросы;
- не кэширует результаты запросов.

Поиске IP-адреса сайта по его доменному имени выполняется в несколько этапов.

1. DNS-клиент, желающий послать сообщение по *известному доменному имени*, проверяет наличие соответствующего IP-адреса в локальном кеше. Если IP-адрес найден, то он используется в качестве адреса назначения в заголовке формируемого сообщения (пакета). На этом работа DNS заканчивается.
2. Если в кеше не найден соответствующий IP-адрес, запрос отправляется на внешний **рекурсивный DNS-сервер**, IP-адрес которого указан в настройках сетевого адаптера компьютера, либо может быть получен автоматически от Интернет-провайдера по протоколу DHCP. (*IP-адрес рекурсивного DNS-сервера можно узнать с помощью команды «nslookup» в командной строке Windows*).
3. **Рекурсивный DNS-сервер** также ищет в своей базе IP-адрес, соответствующий *доменному имени*, и если не находит, то последовательно:
 - формирует запрос к одному из **корневых DNS-серверов** для определения *IP-адреса TLD-сервера*, который отвечает за доменную зону верхнего (первого) уровня;
 - посылает запрос к **TLD-серверу доменной зоны**, чтобы получить *IP-адрес авторитативного DNS-сервера*, содержащего полную информацию о запрашиваемой доменной зоне;
 - формирует запрос к **авторитативному DNS-серверу** о запрашиваемой доменной зоне для получения искомого IP-адреса, соответствующего *доменному имени*;
 - если искомый IP-адрес найден, то он отсылается DNS-клиенту; в противном случае клиенту отправляется сообщение DNS_PROBE_FINISHED_NXDOMAIN (адрес не найден).
4. DNS-клиент, получив IP-адрес кеширует его для дальнейшего использования.
5. Используя этот IP-адрес DNS-клиент посылает сообщение.

Эффективность и результативность протокола DNS обеспечивается за счет:

- распределённого администрирования;
- распределённого хранения данных;
- кэширования информации;
- иерархической структуры;
- резервирования.

Протокол DNS в качестве транспорта может использовать TCP или UDP и порт 53. Обычно небольшие запросы и ответы формируются в виде одной UDP-дейтаграммы. TCP используется в том случае, если размер данных превышает 512 байт.

Слайд 21**Фрагментация IP-пакетов**

В объединяемых сетях разных технологий допустимая максимальная длина пакетов (**Maximum Transfer Unit, MTU**) различна и варьируется от 53 байт в АТМ-сетях до 65 535 байт в IP-сетях. При объединении таких сетей возникает проблема, связанная необходимостью разбиения большого пакета при его передаче через сеть с меньшей допустимой длиной пакета. Если пакет проходит через последовательность сетей и попадает в сеть, у которой значение MTU оказывается меньше размера пакета, пограничный маршрутизатор разбивает пакет на две или более части.

Процесс разбиения длинного пакета на более короткие называется **фрагментацией**, а соответствующие короткие пакеты – **фрагментами**. При фрагментации каждый новый пакет получает свой IP-заголовок (20 байт), что увеличивает накладные расходы. После прохождения фрагментов через соответствующую сеть необходимо восстановить исходный пакет из фрагментов.

Фрагментация в сетях может быть реализована двумя способами:

- прозрачная фрагментация;
- сквозная фрагментация.

Принцип реализации **прозрачной фрагментации** на примере передачи длинного пакета от узла А к узлу В через две подсети (Ethernet и АТМ) с меньшим значением MTU показан на слайде.

Подсети с разными MTU имеют шлюзы – специализированные маршрутизаторы, предоставляющие интерфейсы для связи с другими сетями. Если на такой шлюз приходит пакет слишком большого размера, он разбивается на фрагменты в соответствии с принятым в данной сети значением MTU. Каждый фрагмент адресуется одному и тому же выходному шлюзу, который восстанавливает из этих фрагментов исходный пакет. Таким образом, прохождение данных через сети (подсети) с маленькими значениями MTU оказывается прозрачным для пользователей.

Прозрачная фрагментация обладает простотой, но при этом имеет ряд существенных недостатков:

- выходной шлюз должен собрать все фрагменты для восстановления исходного пакета, для чего в заголовках фрагментов необходимо иметь дополнительную информацию, например, номер фрагмента и признак последнего фрагмента;
- все фрагменты одного пакета должны покидать подсеть через один и то же шлюз, что снижает эффективность маршрутизации;
- появляются дополнительные накладные расходы на фрагментацию и дефрагментацию, что снижает производительность сети и увеличивает время доставки пакетов.

Сквозная фрагментация является альтернативной по отношению к прозрачной фрагментации и состоит в отказе от восстановления пакета из

фрагментов в каждой подсети. Пакет разбивается на фрагменты сразу же в узле-отправителе А или в шлюзе Ш₁ сети. Эти фрагменты передаются по сети как самостоятельные пакеты независимо друг от друга и собираются только в конечном шлюзе Ш_N или узле-получателе В.

Недостатками такого способа фрагментации являются следующие:

- необходимо, чтобы каждый узел (или шлюз) могли восстанавливать пакеты из фрагментов;
- возрастают накладные расходы на передачу данных, так как каждый фрагмент должен иметь заголовок, который сохраняется на протяжении всего пути, что снижает пропускную способность сети;
- необходимо иметь информацию о том, какие значения MTU имеют подсети, через которые проходит путь передачи данных, чтобы задать размер фрагментов.

Для того чтобы правильно восстановить исходный пакет из фрагментов необходимо иметь эффективную систему нумерации фрагментов. Одна из таких систем основана на понятии *элементарного фрагмента*, имеющего небольшой размер, достаточный для его передачи через любую подсеть. Например, длина элементарного фрагмента может быть равна 8 байтам. Исходный пакет разбивается на множество элементарных фрагментов одинаковой длины, кроме последнего, который может быть короче. Фрагменты, формируемые в некоторой подсети и называемые *межсетевыми пакетами*, могут состоять из нескольких элементарных фрагментов, число которых определяется значением MTU, принятым для данной подсети. Заголовок таких фрагментов должен содержать:

- номер исходного пакета (ИП);
- номер первого элементарного фрагмента (нумерация начинается с нуля), содержащегося в нём, который в заголовке IP-пакета называется *смещением фрагмента* (СФ);
- признак конца (ПК) пакета.

Поскольку размер элементарного фрагмента выбирается таким образом, чтобы он мог пройти через любую сеть, дальнейшая фрагментация межсетевого пакета не составляет проблемы.

Слайд 22

Транспортные протоколы TCP/IP: общие принципы

Транспортные протоколы TCP и UDP стека протоколов TCP/IP обеспечивают передачу данных между любой парой *прикладных процессов*, выполняющихся в сети, и предоставляют интерфейс для протокола IP путем демультиплексирования нескольких процессов, использующих в качестве адресов транспортного уровня порты. Для каждого прикладного процесса (ПП) (приложения), выполняемого в компьютере, может быть сформировано *несколько точек входа*, выступающих в качестве *транспортных адресов*, называемых *портами*.

Существуют два способа присвоения порта приложению:

- **централизованный** (присвоенные или назначенные номера от 0 до 1023), использующий стандартные номера, присвоенные общедоступным службам (приложениям), например: FTP – 21, telnet – 23, SMTP – 25, DNS – 53, HTTP – 80.

- **локальный** (динамические номера от 1024 до 65535), предоставляющий произвольный номер из списка свободных номеров при поступлении запроса от приложения пользователя.

Динамические номера портов приложений являются уникальными в пределах каждого компьютера, но могут совпадать с номерами портов в других компьютерах. Различие между ними определяется только различием интерфейсов каждого из компьютеров, задаваемых IP-адресами. Таким образом, пара «**IP-адрес; номер порта**», называемая **сокетом** (socket), однозначно определяет прикладной процесс в сети.

Номера UDP- и TCP-портов в пределах одного и того же компьютера могут совпадать, хотя и идентифицируют разные приложения. Поэтому при записи номера порта обязательно указывается тип протокола транспортного уровня, например 2345/TCP и 2345/UDP. В некоторых случаях, когда приложение может обращаться по выбору к протоколу UDP или TCP, ему могут быть назначены одинаковые номера UDP- и TCP-портов, например DNS-приложению назначен номер 53 – 53/UDP и 53/TCP.

Слайд 23

Протокол UDP

UDP – транспортный протокол, обеспечивающий передачу данных в виде *дейтаграмм* между любой парой *прикладных процессов*, выполняющихся в сети, *без установления соединения*. Дейтаграммы состоят из 8-байтового заголовка, за которым следует поле данных. Заголовок UDP-дейтаграммы показан на слайде.

Наиболее широко UDP используется при выполнении клиент-серверных приложений (типа запрос-ответ).

При этом UDP:

- не контролирует поток,
- не контролирует ошибки в процессе передачи данных,
- не повторяет передачу после получения испорченной дейтаграммы.

Примерами приложений, использующих протокол UDP для передачи данных, являются DHCP, DNS, SNMP.

В некоторых случаях на одном конечном узле может выполняться несколько копий одного и того же приложения. Возникает вопрос: каким образом различаются эти приложения?

Для этого рассмотрим на простом примере процесс формирования запроса и процедуру обращения DNS-клиента к DNS-серверу, когда на одном компьютере запущены два DNS-сервера, причём оба используют для передачи своих данных транспортный протокол UDP. Для того чтобы различать DNS-серверы, им присваиваются разные IP-адреса – IP1 и IP2, которые вместе с

номером порта образуют два разных сокета: «UDP-порт 53, IP1» и «UDP-порт 53, IP2».

Рисунок на слайде иллюстрирует процесс формирования DNS-клиентом запроса к DNS-серверу.

DNS-запрос транспортном уровне стека протоколов TCP/IP передаётся протоколу UDP, который вкладывает этот запрос в UDP-дейтаграмму и указывает в заголовке порт назначения 53/UDP. Затем UDP-дейтаграмма передаётся на межсетевой уровень, где она вкладывается в IP-пакет, заголовок которого содержит «IP-адрес: IP2». IP-пакет, в свою очередь, передаётся на уровень «межсетевой интерфейс», где он помещается в кадр канального уровня с соответствующим заголовком канального уровня (ЗКУ). Этот кадр передаётся по сети к компьютеру, содержащему два DNS-сервера.

В этом компьютере протокол канального уровня (ПКУ) снимает заголовок ЗКУ и передаёт содержимое кадра на межсетевой уровень протоколу IP, который, в свою очередь, извлекает содержимое (UDP-дейтаграмму) из IP-пакета. Дальнейшие манипуляции с передаваемыми данными отличаются от принципов, заложенных в многоуровневую модель иерархии протоколов. Вместо того чтобы просто передать UDP-дейтаграмму, находящуюся в поле данных IP-пакета, транспортному уровню, *IP-протокол присоединяет к UDP-дейтаграмме* так называемый **псевдозаголовок**, содержащий среди прочего IP-адреса отправителя и получателя. Таким образом, протокол UDP, имея IP-адрес и порт назначения, однозначно определяет, что содержимое поля данных (то есть DNS-запрос), должно быть передано приложению «DNS-сервер 2».

Псевдозаголовок используется в UDP-дейтаграммах и в TCP-сегментах.

Слайд 24

Псевдозаголовок протоколов UDP и TCP

Как сказано выше, при передаче данных от нижележащего межсетевого уровня на транспортный уровень в заголовки UDP-дейтаграмм и TCP-сегментов включается псевдозаголовок, который располагается перед основным заголовком транспортного уровня. Таким образом, блок данных транспортного уровня (UDP-дейтаграмма или TCP-сегмент) будет содержать:

- псевдозаголовок длиной 12 байт (три 32-разрядных слова);
- заголовок длиной 8 байт для UDP-дейтаграммы или 20 и более байт для TCP-сегмента;

- данные.

Псевдозаголовок, формат которого показан на слайде, содержит:

- IP-адрес отправителя;
- IP-адрес получателя;
- нулевое поле, не используемое и заполненное нулями;
- поле «Протокол», содержащее номер протокола транспортного уровня: 17 для протокола UDP и 6 для протокола TCP;
- длина UDP-дейтаграммы или TCP-сегмента.

Включение псевдозаголовка в контрольную сумму блока данных транспортного протокола помогает обнаружить неверно доставленные пакеты за счёт двойной проверки, выполняемой протоколом IP и протоколами транспортного уровня. Кроме того, передача IP-адресов транспортному уровню позволяет однозначно разрешить ситуацию, когда две копии одного и того же приложения используют одинаковый номер порта.

Узел-отправитель при формировании TCP-сегмента рассчитывает контрольную сумму сегмента с учётом псевдозаголовка. Однако при передаче по сети *псевдозаголовок не включается в сегмент*, что позволяет уменьшить накладные расходы и, соответственно, повысить эффективную скорость передачи пользовательских данных. В узле-получателе протокол IP формирует псевдозаголовок и вставляет его в поступивший сегмент и передаёт транспортному уровню.

Слайд 25

Протокол TCP: принципы реализации

1. Протокол TCP обеспечивает **надёжную передачу** данных между прикладными процессами за счёт установления логических соединений между взаимодействующими процессами.

Логическое соединение между двумя прикладными процессами идентифицируется парой сокетов (IP-адрес, номер порта), каждый из которых описывает один из взаимодействующих процессов.

2. В процессе установления логического соединения отправитель и получатель **согласовывают параметры процедуры обмена** данными, основными среди которых являются:

- *начальный порядковый номер байта* ISN (Initial Sequence Number), представляющий собой случайное число в интервале $(0; 2^{32})$;
- *максимальный размер передаваемых сегментов*;
- *максимальный объем данных*, который получатель разрешает передать отправителю без подтверждения (максимальный размер окна).

3. Информация, поступающая к протоколу TCP в рамках логического соединения от протоколов более высокого уровня, рассматривается протоколом TCP как *неструктурированный поток байтов* и заносится в буфер. Для передачи на сетевой уровень из буфера вырезается **сегмент**, не превосходящий 64 Кбайт (максимального размера IP-пакета). На практике обычно длина сегмента ограничивается значением 1460 байтами, что позволяет поместить его в поле данных (1500 байт) кадра Ethernet с заголовками TCP (20 байт) и IP (20 байт).

Соединение TCP ориентировано на дуплексную передачу, причем если одна из сторон не имеет данных для передачи, она формирует и передает положительные квитанции в рамках механизма «скользящего окна», содержащие только заголовки сегмента (без поля данных).

4. **Управление потоком** данных в протоколе TCP осуществляется с использованием механизма **скользящего окна переменного размера**.

5. При передаче сегмента узел-отправитель включает таймер и ожидает подтверждения в течение **тайм-аута**.

6. В протоколе ТСП используются *только положительные квитанции* – подтверждения АСК (от англ. Acknowledgment), посылаемые отправителю получателем. Отрицательные квитанции не посылаются, а для обеспечения гарантии доставки данных используется *механизм тайм-аута*. Узел назначения, получивший сегмент формирует и посылает обратно сегмент (с данными, если они есть, или без данных) с номером подтверждения, равным следующему порядковому номеру ожидаемого байта.

7. В отличие от многих других протоколов, протокол ТСП подтверждает получение *не сегментов, а байтов*. Если время ожидания подтверждения истекает, отправитель посылает сегменты еще раз.

8. Контрольная сумма (заголовок + данные + псевдозаголовок) рассчитывается в *дополнительном коде* для 16 разрядных слов.

При передаче каждого сегмента некоторого сообщения отправитель в соответствующем поле заголовка сегмента указывает номер первого байта данных, определяющий смещение этого сегмента относительно начала сообщения. Номер первого байта первого сегмента каждого сообщения формируется случайным образом *по специальному алгоритму*.

Положим, что между отправителем и получателем данных соединение установлено, тогда последовательность действий при **передаче сегментов** будет выглядеть следующим образом:

- отправитель посылает первый сегмент с данными, содержащий первые 1460 байтов передаваемого сообщения, с указанием в заголовке *номера первого байта*, формируемого по определенному алгоритму в виде некоторого случайного числа S , называемого **начальным последовательным номером (ISN, Initial Sequence Number)**;
- если сегмент принят без ошибок, получатель отправляет сегмент с подтверждением (в заголовке флаг АСК=1), в заголовке которого в соответствующем поле указывает номер следующего ожидаемого байта ($S+1460$);
- отправитель передает следующий сегмент, содержащий следующие 1460 байтов, включая в его заголовок *номер первого байта в сегменте* ($S+1460$);
- получатель отправляет подтверждение, что он *ждет байт с номером* ($S+2920$) и т.д.;
- если сегменты придут в неправильном порядке, то получатель по номерам байтов сможет собрать их в правильной последовательности.

Существуют два **типа подтверждения**, которые могут использоваться совместно с алгоритмом скользящего окна:

- **кумулятивное** (по умолчанию) – подтверждение приема указанного байта и всех предыдущих;

- **выборочное** – подтверждение диапазона принятых байтов при большом размере окна (используется дополнительное поле заголовка ТСП).

Кумулятивное подтверждение свидетельствует о том, что получен указанный байт данных и все предыдущие. В ТСП такое подтверждение используется *по умолчанию*.

В то же время использование высокоскоростных каналов связи большой протяженности позволяет увеличить размер окна до 1 Гбайта. При этом, если при передаче гигабайта данных потерялся всего лишь один сегмент, который находится примерно в начале, отправителю поступит кумулятивное подтверждение о получении, например, первых 300 Мбайт, что потребует повторной передачи следующих 700 Мбайт данных, которые уже есть у получателя.

Устранение этой проблемы достигается применением **выборочного подтверждения**, при котором получатель подтверждает получение диапазона принятых байт. Он получил первые 300 Мбайт и вторые 700 Мбайт из гигабайта и не получил всего лишь один сегмент, находящийся между ними. Отправитель вместо вторых 700 мегабайт, повторно передает всего лишь один недостающий сегмент. Выборочное подтверждение эффективно при большом размере окна ТСП, но в ТСП, как сказано выше, по умолчанию используется кумулятивное подтверждение.

Для реализации выборочного подтверждения используются *дополнительные поля заголовка ТСП – параметры*.

Слайд 26

Протокол ТСП: проблемы реализации

Несмотря на кажущуюся простоту протокола, в нем имеется ряд нюансов, которые могут привести к некоторым проблемам.

Во-первых, **сегмент или подтверждение ACK=1 могут быть потеряны, искажены или продублированы**. В этих случаях, как показано на рисунках, используется механизм тайм-аута, после которого отправитель повторно отправляет этот же сегмент.

Может возникнуть ситуация, когда получателем сегмент с номером первого байта 1460 был принят правильно, но подтверждение с номером ожидаемого байта 2920 не дошло до отправителя, и отправитель по истечении тайм-аута посылает тот же сегмент с тем же номером первого байта повторно. Для исключения дублирования сегментов получатель анализирует номер первого байта. Если у получателя есть сегмент с таким же номером 1460, он его отбрасывает и повторно отправляет подтверждение с указанием ожидаемого байта 2920.

Во-вторых, поскольку **сегменты при передаче по сети могут фрагментироваться**, возможна ситуация, при которой часть переданного сегмента будет принята, а остальная часть окажется потерянной.

В-третьих, **сегменты могут прибывать в узел назначения в произвольном порядке**, что может привести к ситуации, при которой байты с 2920 по 4379 уже прибыли, но подтверждение для них не может быть выслано, так как байты с 1460 по 2919 еще не получены.

В-четвертых, **сегменты могут задержаться в сети так долго, что у отправителя истечёт интервал ожидания**, и он передаст их снова. Переданный повторно сегмент может:

- пройти по другому маршруту;
- быть иначе фрагментирован;
- попасть в перегруженную сеть.

В результате для восстановления исходного сегмента потребуется достаточно сложная обработка.

В-пятых, **скорость передачи данных отправителя может превышать возможности получателя**, что приводит к возникновению перегрузок и к значительному увеличению задержек сегментов и даже к их потере. Для борьбы с перегрузками применяются специальные методы управления трафиком для снижения негативного эффекта от перегрузок.

В-шестых, сеть, а точнее маршрутизаторы и каналы на пути передачи сегментов могут быть **перегружены**, что требует применения специальных методов *контроля и борьбы с перегрузками*.

Слайд 27

Протокол TCP: основные функции

На слайде представлена укрупненная схема организации передачи и приема данных на основе протокола TCP, включающая следующие основные функции:

- **при передаче данных:**
 - *установление соединения* по запросу от приложения путем формирования и передачи адресату запроса на установление соединения;
 - *прием в буфер передачи сообщения от приложения* при установленном дуплексном соединении;
 - *разбиение сообщения на сегменты* определенной длины, согласованной в процессе установления соединения; в большинстве случаев размер сегмента принимается равным 1460 байтам в предположении, что на канальном уровне IP-пакеты, содержащие TCP-сегменты, будут вкладываться в Ethernet-кадры;
 - *формирование TCP-заголовков* в соответствии с требованиями, предъявляемыми к качеству передачи данных со стороны приложения;
 - *передача сегментов протоколу IP*, который формирует IP-пакеты, присоединяя IP-заголовки, и передает их далее протоколу канального уровня, например протоколу Ethernet.

- **при приеме данных:**

- *подтверждение соединения* на начальном этапе установления логического дуплексного соединения по инициативе узла-отправителя;
- *прием сегментов от протокола IP* и их запись в буфер приема;
- *анализ TCP-заголовков* и реализация функций, предписанных значениями соответствующих полей заголовка;
- *формирование сообщения* путем объединения сегментов;
- *передача сообщения приложению.*

Слайд 28

Протокол TCP: формат заголовка TCP-сегмента

Максимальный размер TCP-сегмента определяется исходя из максимального размера IP-пакета, равного 65 535 байтам, что с учетом минимально возможной длины IP-заголовка в 20 байт, составляет 65515 байт.

Первоначальный формат TCP-сегмента, показанный на слайде, содержал в поле «**Резерв**» шесть неиспользуемых битов, из которых в дальнейшем 3 бита стали использоваться в качестве признаков для управления и борьбы с перегрузками.

Первые 20-байт заголовка имеют строго фиксированный формат, за которым могут находиться дополнительные поля с **параметрами**, значения которых согласовываются взаимодействующими сторонами *в процессе установления соединения*. После дополнительных полей заголовка размещается поле данных, содержащее не более 65 495 байт, которое вместе с TCP- и IP-заголовками размером по 20 байт даст максимально допустимый размер IP-пакета в 65 535 байт.

Рассмотрим назначение фиксированных полей заголовка TCP-сегмента.

Поля «**Порт отправителя**» (2 байта) и «**Порт получателя**» (2 байта) содержат *номера портов* приложений, между которыми установлено логическое соединение.

Поле «**Порядковый номер**» (4 байта) содержит *номер первого байта* данных в передаваемом сегменте, который определяет смещение этого сегмента относительно потока передаваемых данных, что позволяет принимающей стороне контролировать порядок сегментов и корректно восстановить всё сообщение.

В каждом TCP-соединении поддерживается свой уникальный «порядковый номер» для отправляемых сегментов в интервале от 0 до $4\,294\,967\,295\ (2^{32}-1)$.

Для исключения конфликтов в случае совпадения порядковых номеров в сегментах из разных TCP-соединений *начальное значение* «порядкового номера» (ISN - Initial Sequence Number), указываемое в процессе установления TCP-соединения в сегменте с флагом SYN=1 рассчитывается следующим образом. Значение ISN увеличивается на единицу за каждые прошедшие 4 микросекунды

и на 64 000 – каждый раз при установлении нового TCP-соединения. Легко убедиться, что сгенерированное число будет уникальным для периода времени почти в пять часов. Первый байт полезных данных в устанавливаемой сессии будет иметь номер $SN=ISN+1$.

После установления соединения при отправке последующих сегментов, значение порядкового номера SN увеличивается на 1 для каждого байта отправленных данных, а также для всех сегментов с флагом SYN и FIN. Это обеспечивает принимающей системе обработку сегментов в той последовательности, как они были сформированы при отправлении.

Поле «**Подтвержденный номер**» или «Номер подтверждения» (4 байта) содержит *номер следующего ожидаемого байта* на приемной стороне при использовании получателем TCP-сегмента в качестве положительной квитанции (флаг ACK=1), подтверждающей правильный приём всех предыдущих байтов.

Каждая сторона подсчитывает свой «порядковый номер» для переданных данных и «подтвержденный номер» для полученных данных, причем «порядковому номеру» одной стороны соответствует «подтвержденный номер» другой стороны.

Поле «**Длина заголовка**» (4 бита) задаёт длину заголовка TCP-сегмента, измеренную в *32-битовых словах*.

Минимальный размер заголовка TCP-сегмента составляет 20 байт, а максимальный – 60 байт. Длина заголовка определяет смещение полезных данных относительно начала сегмента.

Поле «**Резерв**» длиной 6 бит первоначально было зарезервировано. В настоящее время 3 бита в этом поле используются в качестве флагов для управления и борьбы с перегрузками, возникающими в сети.

Однобитовые **флаги** (кодовые биты) несут служебную информацию о типе сегмента и интерпретируются следующим образом:

URG=1 (от слова **urgent** – срочный) указывает на наличие *срочных данных*, что означает использование поля «Указатель на срочные данные».

ACK=1 (от слова **acknowledgement**) означает, что сегмент является *положительной квитанцией* (подтверждением) на принятый сегмент и поле «Подтвержденный номер» содержит номер ожидаемого байта. Все сегменты после стартового сегмента с SYN=1 будут иметь установленный флаг ACK=1. Если сегмент не используется как подтверждение (ACK=0), поле «Номер подтверждения» просто игнорируется.

PSH=1 (PUSH-флаг) означает *запрос на отправку данных* без ожидания заполнения буфера. При нормальной передаче данных получатель не подтверждает получение каждого сегмента, а в течение некоторого времени собирает и хранит полученные сегменты в буфере, прежде чем передать их приложению пользователя. Сегмент с флагом PUSH=1 предписывает получателю немедленно передать приложению пользователя все полученные сегменты из буфера и послать отправителю подтверждение.

RST=1 (от слова *reset* – сброс) используется для *сброса состояния соединения* при обнаружении проблем, ошибок и нестандартных ситуаций, а также для отказа от неверного сегмента или от попытки создать соединение.

SYN=1 (от слова *syncing* – синхронизация) используется для *установки соединения*, при этом если ACK=0, то это означает, что поле подтверждения не используется;

FIN=1 (от слова *finish* – финиш) используется для *разрыва соединения*.

Поле **«Размер окна»** (2 байта) определяет, сколько байт может быть послано после байта, получившего подтверждение.

Поле **«Контрольная сумма»** (2 байта) содержит контрольную сумму, которая охватывает заголовок, данные и **псевдозаголовок**.

Поле **«Указатель на срочные данные»** (2 байта) при установленном флаге URG=1 содержит численное значение смещения в байтах от «Порядкового номера». Смещение указывает на последний байт срочных данных в сегменте, которые необходимо немедленно принять, даже если буфер приема переполнен. Например, если сегмент содержит 1000 байт срочных данных, после которых находятся 460 байт обычных данных, будет установлен флаг URG=1, а «Указатель на срочные данные» будет иметь значение 1000. Таким образом в протоколе TCP реализуются *прерывающие сообщения*. Содержимым срочных данных занимается прикладной уровень. Протокол TCP лишь обеспечивает их доставку и не интересуется причиной прерывания.

TCP предоставляет процессу возможность приоритетной отправки данных с использованием флага URG. Процесс, которому необходимо отправить срочные данные, отправляет их протоколу TCP с флагом URG=1.

Протокол TCP создает сегмент с флагом URG=1, и в поле «Указатель на срочные данные» записывает значение смещения, указывающее на последний байт срочных данных в сегменте. Например, если в поле данных сегмента находится 1000 байт срочных данных, за которыми следуют обычные данные, будет установлен флаг URG=1, а в поле «Указатель на срочные данные» будет занесено значение 1000.

При получении сегмента с флагом URG=1 на принимающей стороне по значению в поле «Указатель на срочные данные» определяется, какие данные в сегменте являются срочными, которые затем пересылаются процессу с указанием «срочные». Остальные данные в сегменте обрабатываются обычным способом.

Поле **«Параметры»** имеет переменную длину и содержит параметры, значения которых согласовываются в процессе установления соединения. При передаче данных после установления соединения это поле обычно отсутствует.

Примерами приложений, использующих протокол TCP для передачи данных, являются FTP, TFTP, DNS, POP3, IMAP, TELNET.

Слайд 29**Протокол TCP: флаги для управления перегрузкой**

В последней версии протокола TCP для управления перегрузкой в поле «Резерв» были добавлены еще 3 однобитовых флага:

- ECE (ECN-Echo) выполняет двойную роль в зависимости от значения флага SYN:
 - при установлении соединения (SYN=1) указывает, что отправитель поддерживает ECN;
 - при передаче данных (SYN=0) указывает отправителю о перегрузке (или надвигающейся перегрузке) в сети;
- CWR (Congestion Window Reduced) – «окно перегрузки уменьшено» – устанавливается отправителем для подтверждения получения сегмента с флажком ECE от узла-получателя и включения отправителем механизма уменьшения перегрузки (Congestion Control);
- NS (Nonce Sum) – «одноразовая сумма» – для улучшения работы механизма *явного уведомления о перегрузке* (Explicit Congestion Notification, ECN).

В качестве дополнительных параметров в заголовке сегмента TCP могут указываться:

1) максимальный размер сегмента MSS (Maximum Segment Size), который устанавливается и согласовывается отправителем и получателем при установке соединения (для Ethernet - 1460 байт);

2) масштаб окна, позволяющий увеличить размер окна до 1 ГБ вместо 65535 байт в случае высокоскоростных каналов связи между отправителем и получателем;

3) выборочное подтверждение диапазона принятых байт, а не всех данных, если потерян всего лишь один сегмент или небольшая часть в большом потоке данных.

Слайд 30**Протокол TCP: вычисление контрольной суммы**

Алгоритм вычисления контрольной суммы выглядит следующим образом.

Перед началом вычисления контрольной суммы отправителем сегмента значение этого поля устанавливается равным нулю.

Если поле данных содержит нечётное число байтов, то оно дополняется нулевым байтом, который используется при подсчёте контрольной суммы, но не вставляется в сегмент для передачи в сети. Необходимость такого добавления обусловлена тем, что TCP-сегмент, включающий *заголовок, данные и псевдозаголовок*, рассматривается как совокупность 16-разрядных двоичных чисел, которые складываются. Если длина N суммы превышает 16 разрядов, то

($N-16$) старших разрядов суммы рассматриваются как младшие разряды 16-разрядного двоичного числа (путем добавления нулей в старшие разряды), которое добавляется к этой сумме. Затем окончательно полученная сумма инвертируется (переводится в дополнительный код) и заносится в поле «Контрольная сумма».

Получатель сегмента аналогичным образом подсчитывает контрольную сумму для всего сегмента, включая поле «Контрольная сумма». Очевидно, что полученный таким образом результат должен быть равен 0.

Отметим, что *дополнительный нулевой байт и псевдозаголовок не передаются по сети.*